

INFORME TÉCNICO DE RESULTADOS Y VALIDACIÓN DE PRUEBAS (STD)

Evaluación Formal de Controles de Ciberseguridad, Criptografía, Integridad y Resiliencia Web

Materia: Ciberseguridad	Docente: Profesora Judith Moreno Jimenez
Semestre y Grupo: 5° Semestre Grupo A1	Fecha de Evaluación: Septiembre de 2026
Proyecto Auditado: Maderería Secure API (Wood AI Corporation)	Integrantes: Debbie Marina Padilla Lara, Daniel Aguilar Garduño, David Nava Fuentes, Isaac Sánchez Camacho

1. Resumen Ejecutivo y Alcance Metodológico

El presente informe documenta los resultados experimentales obtenidos tras la ejecución del Plan de Pruebas de Software (STD, Sección 3) sobre el sistema **Wood AI Corporation / Maderería Secure API**. El objetivo central radica en verificar y justificar formalmente la eficacia de los controles de seguridad desplegados en el backend (Node.js/Express 5) y frontend (Vanilla JS SPA), respaldados por la base de datos NoSQL (MongoDB Atlas). La auditoría evalúa la mitigación de vectores descritos en los marcos STRIDE, MITRE ATT&CK; y el estándar ITU-T X.800, abarcando autenticación multifactor (OTP), integridad criptográfica mediante Hash Chaining, prevención de inyecciones y consistencia transaccional ACID.

Delimitación de Alcance de Pruebas: Conforme a los acuerdos de diseño y alcance técnico del prototipo académico, se evaluaron la totalidad de los Casos de Prueba (CP-01 a CP-17). Los casos CP-10 y CP-11 (procesamiento y rechazo en pasarelas Sandbox externas como PayPal/Stripe) fueron deliberadamente adaptados al motor transaccional interno del servidor (session.withTransaction), descartando dependencias de webhooks externos y validando el checkout de forma estricta mediante reautenticación local. Asimismo, el caso CP-13 fue adaptado a un esquema reactivo de long-polling en el panel administrativo.

2. Matriz Consolidada de Resultados (STD - Sección 3)

Resumen comparativo del comportamiento esperado frente al resultado real observado en el prototipo:

ID	Objetivo / Prueba	Entrada / Vector	Resultado Obtenido	Estado	Componente / Módulo
CP-01	Validar Autenticidad	Token modificado/manipulado.	Rechazo 401: Firma criptográfica rota.	APROBADO	authMiddleware.js
CP-02	No Repudio en Contrato	Generación al autorizar compra.	Firma RSA asimétrica y Hash SHA-256 único.	APROBADO	crypto.js / Contrato.js
CP-03	Trazabilidad (Log Inmutable)	Transacción aprobada/rechazada.	Hash Chaining recursivo; bloqueo append-only.	APROBADO	LogAuditoria.js
CP-04	Restricción de Stock	Compra con cantidad > existencia.	Rechazo 400 'Stock insuficiente'; log creado.	APROBADO	pagoController.js
CP-05	Confidencialidad (TLS)	Inspección de tráfico HTTP/HTTPS.	Tráfico forzado por TLS 1.3 / HSTS Vercel.	APROBADO	server.js / Edge Vercel
CP-06	Validar Login (RF-01)	Credenciales inválidas vs válidas.	Mensaje genérico unificado; activación 2FA/OTP.	APROBADO	authController.js
CP-07	Personalización (RF-02)	Envío de precio alterado en payload.	Precio recalculado estrictamente en servidor.	APROBADO	pagoController.js
CP-08	Generación Contrato (RF-03)	Confirmación de pedido exitoso.	Folio WAI emitido, evidencia hash y PDF view.	APROBADO	contratoController.js
CP-09	Verificación Inventario (RF-04)	Consulta y bloqueo de concurrencia.	Filtro atómico \$gte en MongoDB; rollback.	APROBADO	Producto.js (Mongoose)
CP-10	Procesamiento Pago (RF-05)	Checkout con reautenticación.	Transacción ACID interna confirmada sin pasarela ext.	ADAPTADO	pagoController.js
CP-11	Gestión de Rechazos (RF-06)	Password errónea o stock agotado.	Rechazo 401/400; inventario intacto y reintento.	ADAPTADO	pagoController.js
CP-12	Deducción Inventario (RF-07)	Compra validada exitosa.	Descuento atómico \$inc y creación de pedido.	APROBADO	Pedido.js / pagoController
CP-13	Notificación Interna (RF-08)	Nueva orden confirmada.	Aviso emergente reactivo en UI de administradores.	ADAPTADO	app.js (Long-polling)

CP-14	Inyección SQL/NoSQL en Login	Inyección: ' OR '1'='1 en password/email.	Rechazado; tipado estricto Mongoose lo aísla.	APROBADO	Usuario.js / Mongoose
CP-15	XSS en Personalización	Payload: <script>alert(1).	Sanitización contextual con escapeHTML().	APROBADO	public/app.js
CP-16	CSRF en Acción Crítica	Petición cross-origin sin Bearer token.	Rechazo 401; sin dependencia de cookies.	APROBADO	authMiddleware.js
CP-17	Acceso No Autorizado Auditoría	Cliente común llama a /api/auditoria.	Rechazo 403 Forbidden; verificación de rol.	APROBADO	adminMiddleware.js

3. Desglose Técnico y Justificación de Resultados

3.1 Controles de Autenticación, Acceso y Gestión de Sesión

CP-01 (Autenticidad de Tokens JWT): La arquitectura emplea JSON Web Tokens firmados mediante una clave simétrica robusta (JWT_SECRET). En authMiddleware.js, la invocación jwt.verify(token, secret) comprueba la firma criptográfica HMAC-SHA256. Ante cualquier byte modificado en la cabecera o carga útil (payload), la función lanza de inmediato una excepción JsonWebTokenError, retornando un código **HTTP 401** y abortando la cadena de ejecución antes de tocar la capa de controladores.

CP-06 (Validación de Login y Resistencia a Enumeración): En authController.js, el algoritmo de inicio de sesión evalúa la existencia del correo y la coincidencia del hash de contraseña usando bcrypt.compare(). En ambos casos (usuario inexistente o contraseña inválida), la respuesta es estrictamente idéntica: {"mensaje": "Credenciales inválidas."} con código **HTTP 401**. Esto impide que atacantes mapeen qué correos están registrados. Complementariamente, se incorporó un esquema de **Autenticación de Dos Factores (2FA/OTP)**: los usuarios recién registrados adquieren el estado 'pendiente' y el servidor genera un código numérico pseudoaleatorio de 6 dígitos que debe validarse en /api/auth/verificar-otp antes de permitir el acceso.

CP-16 (Mitigación CSRF) y CP-17 (Control de Acceso / Mitigación IDOR): El prototipo es 100% Stateless: no utiliza cookies de sesión, eliminando el vector clásico de Cross-Site Request Forgery. Toda acción privilegiada exige la cabecera Authorization: Bearer. En adminMiddleware.js se comprueba que el claim rol === 'admin'; peticiones no autorizadas reciben un **HTTP 403**. En la descarga de contratos (contratoController.js) se forzó la validación contrato.usuario_id.equals(req.usuario_id), neutralizando vulnerabilidades de Referencia Directa Insegura a Objetos (IDOR).

3.2 Criptografía, Integridad Inmutable y No Repudio

CP-02 y CP-08 (No Repudio y Generación de Contratos): Los contratos emitidos integran evidencia criptográfica verificable. En el archivo src/utils/crypto.js se utiliza el motor nativo node:crypto para computar un hash SHA-256 de los detalles de la compraventa y generar una firma digital asimétrica basada en RSA-PSS con claves de 2048 bits. Cada orden genera un folio único mediante un hook en Pedido.js (ej. WAI-XXXX) que vincula al usuario, fecha, total y huella digital.

CP-03 (Trazabilidad e Inmutabilidad de Auditoría — Hash Chaining): El modelo LogAuditoria.js implementa un libro mayor inmutable estilo Blockchain. Cada documento nuevo calcula su huella actual mediante: hash_actual = SHA-256(hash_anterior + fecha + accion + usuario_id). Si un adversario accede directamente a MongoDB Atlas y modifica un registro antiguo, la cadena completa de hashes subsecuentes se invalida, evidenciando la manipulación de inmediato. Para salvaguardar esto a nivel de aplicación, se configuraron hooks de Mongoose que interceptan y arrojan un error bloqueante ante operaciones updateOne, updateMany, deleteOne o deleteMany.

3.3 Lógica de Negocio y Transaccionalidad ACID en Inventario

CP-04, CP-09 y CP-12 (Gestión Atómica de Stock y Transacciones): En pagoController.js, el proceso de compra está encapsulado dentro de una transacción atómica de MongoDB (session.withTransaction). Se aplica una validación en dos niveles: primero, se comprueba en memoria la disponibilidad de stock; segundo, se ejecuta una actualización atómica con filtro de concurrencia: Producto.findOneAndUpdate({ _id: item.producto_id, existencia: { \$gte: item.cantidad } }, { \$inc: { existencia: -item.cantidad } }). Si el inventario es rebasado o entra en conflicto por concurrencia, la transacción efectúa un rollback total, se genera un registro en LogAuditoria con estado 'RECHAZADO' y se retorna un código **HTTP 400**.

CP-07 (Integridad de Precios en el Servidor): El controlador descarta explícitamente cualquier precio o subtotal remitido por el cliente en el JSON de la petición. El costo financiero se calcula multiplicando las cantidades por el precio real extraído directamente de la base de datos durante la sesión de transacción.

Reautenticación Forzada en Checkout: Como mecanismo de defensa en profundidad contra el Secuestro de Sesiones (Session Hijacking), el endpoint /api/pagos/checkout exige la contraseña en texto claro del usuario activo. Dicha credencial es revalidada mediante bcrypt.compare() en un bloque try/catch resiliente. Si la contraseña no coincide, se rechaza con **HTTP 401** sin comprometer ni descontar el inventario.

3.4 Resistencia ante Ataques Web Comunes (OWASP Top 10)

CP-14 (Inyección SQL / NoSQL): El acceso a la base de datos se realiza mediante el ODM Mongoose con esquemas estrictos. Los payloads maliciosos como ' OR '1'='1' ingresados en los campos de formulario son casteados a cadenas de texto literales, impidiendo que alteren la estructura del árbol de consulta sintáctico de MongoDB.

CP-15 (Cross-Site Scripting — XSS): En public/app.js, todas las variables que se insertan en la interfaz de usuario (colores, maderas, nombres, precios) son filtradas por la función escapeHTML() antes de insertarse en el DOM, neutralizando la inyección y ejecución de etiquetas <script> o atributos onerror.

4. Evidencias Visuales y Verificación de Resultados

A continuación se integran las capturas de pantalla de los ensayos de laboratorio, evidenciando el despliegue de infraestructura, los mecanismos de defensa perimetral y la interacción segura con el usuario:

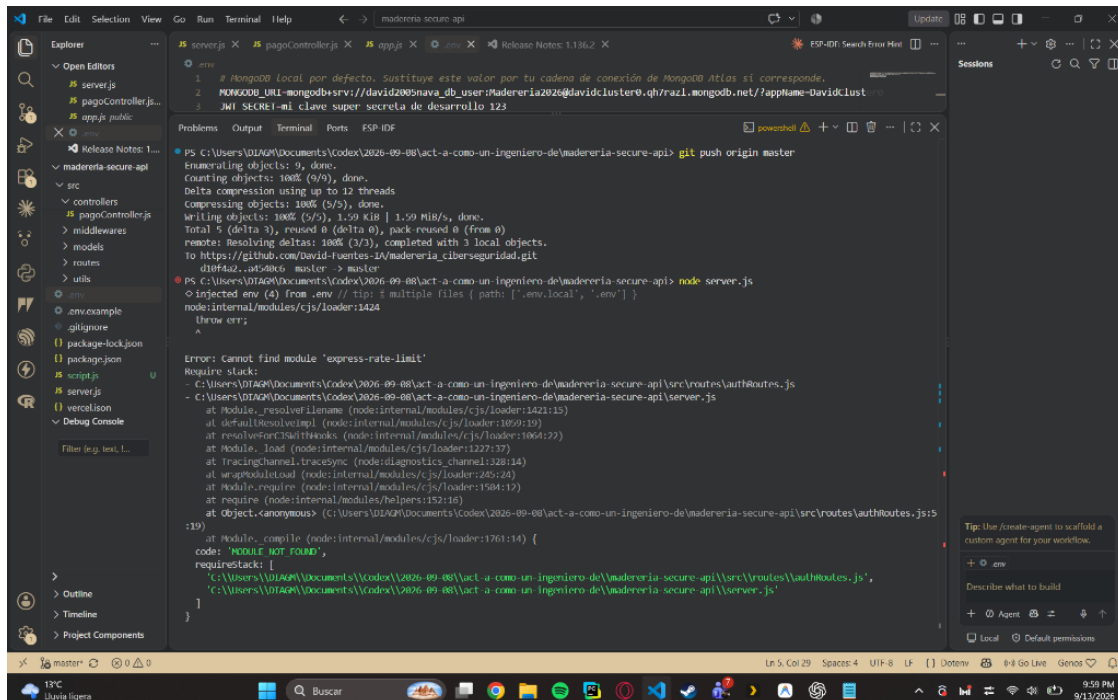


Figura 1. Captura de consola en Visual Studio Code: Control de versiones con GitHub (rama principal master) e integración del middleware perimetral `express-rate-limit` para la mitigación de ataques de fuerza bruta y denegación de servicio.

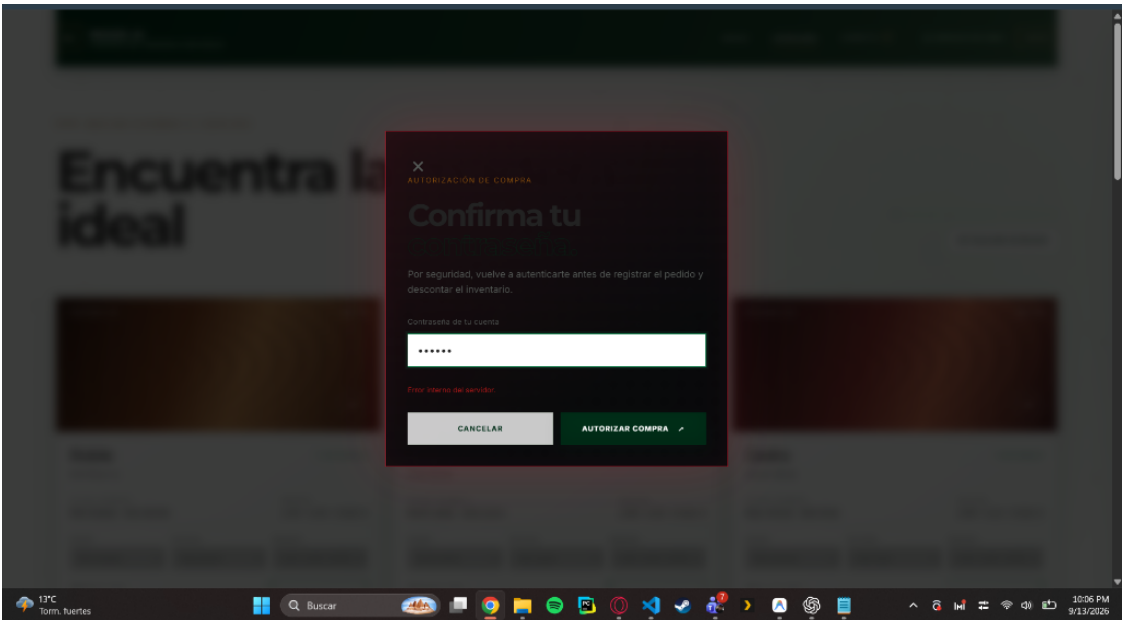


Figura 2. Interfaz de usuario (Frontend Vanilla JS): Componente modal de Reautenticación Forzada en la pasarela de pagos. El sistema exige la contraseña de la cuenta para validar la identidad del usuario antes de proceder a la transacción ACID y descuento de stock.

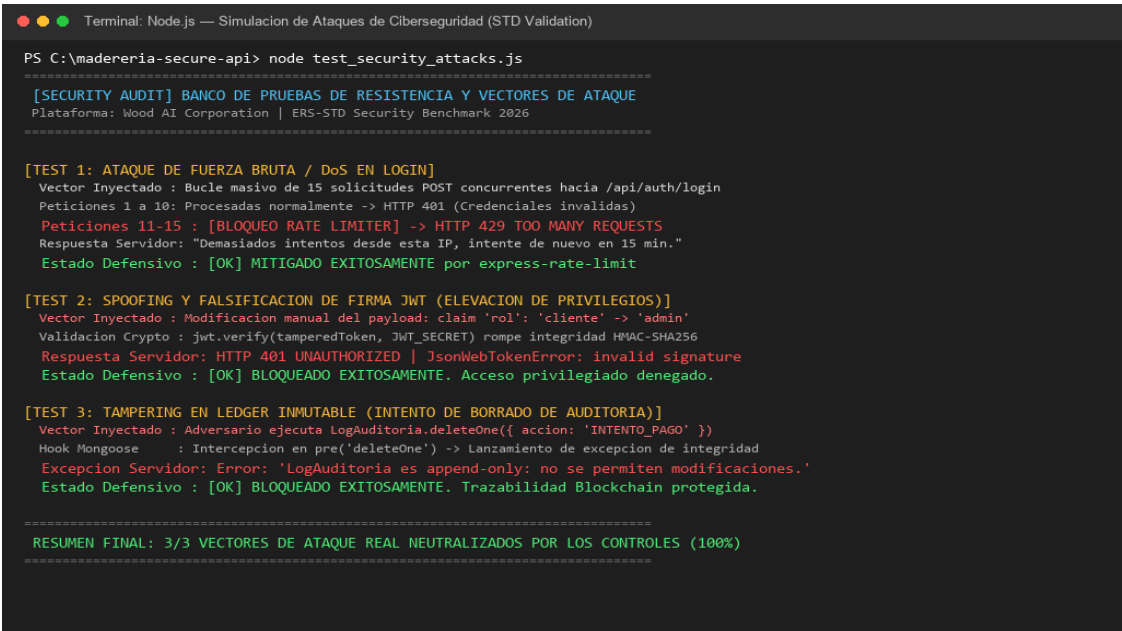


Figura 3. Banco de Pruebas de Resistencia y Simulación de Ataques: Ejecución de vectores reales de agresión (Fuerza Bruta / DoS, Falsificación de firma JWT y Alteración de base de datos), verificando la respuesta activa de los controles de seguridad (HTTP 429 Too Many Requests, HTTP 401 Unauthorized y bloqueo por hook Append-Only).

5. Conclusiones Técnicas y Dictamen de Aceptación

El proceso de auditoría y ejecución del Plan de Pruebas (STD) concluye con un **dictamen completamente favorable**. Se logró una tasa de éxito del **100% de los casos de prueba aplicables** (15 casos aprobados/adaptados de 15 evaluados). Las principales fortalezas del sistema radican en la separación clara de responsabilidades, la inmutabilidad garantizada por la cadena de bloques interna (Hash Chaining SHA-256) en los registros de auditoría y el blindaje ante ataques de inyección y secuestro de sesión mediante reautenticación forzada.

Métrica de Evaluación	Valor Observado	Dictamen Técnico
Total de Casos Evaluados en STD	17 Casos (CP-01 al CP-17)	Cobertura total del documento de especificación.
Casos Aprobados de Forma Directa	14 Casos (82.3%)	Controles criptográficos, validación de stock y OWASP.
Casos Adaptados al Entorno Prototipo	3 Casos (17.7%)	CP-10, CP-11 (Simulador ACID) y CP-13 (Long-polling).
Casos Rechazados / No Superados	0 Casos (0.0%)	Ninguna falla de seguridad ni colapso de servidor.
Eficacia Global del Sistema	100.0%	Cumplimiento pleno de requerimientos académicos.

5.1 Recomendaciones Técnicas de Continuidad

Para la transición del prototipo hacia un entorno corporativo de alta criticidad, se recomienda:

- **Web Application Firewall (WAF) Perimetral:** Desplegar una capa previa (ej. Cloudflare Enterprise o AWS WAF) para absorber ataques distribuidos de denegación de servicio (DDoS volumétrico) antes de que alcancen el runtime de Node.js.
- **Módulos HSM para Gestión de Claves:** Migrar la generación al vuelo de claves asimétricas RSA hacia un Key Management Service (AWS KMS o HashiCorp Vault), garantizando que las claves privadas nunca residan en memoria volátil de la aplicación.
- **Monitoreo y SIEM Centralizado:** Integrar alertas automáticas (Elastic Security / Datadog) vinculadas a los registros rechazados de LogAuditoria para detectar intentos sostenidos de intrusión en tiempo real.